Taller: Taller sobre Patrones de Arquitectura de Microservicios

Descripción: Implementación de Service Discovery y Circuit Breaker en Microservicios

Parte 1: Configuración y Comprensión de Service Discovery

Guía paso a paso:

1. Clonar los repositorios:
    - fx-discovery-server: jrquinte/FutureXEurekaServer (github.com)
    - fx-course-service: jrquinte/FutureXCourseApp (github.com)
    - fx-catalog-service: jrquinte/FutureXCourseCatalog (github.com)
2. Configurar y ejecutar fx-discovery-server:
    - Abrir el proyecto en su IDE
    - Verificar la configuración en application.properties
    - Ejecutar FutureXEurekaServerApplication
3. Configurar y ejecutar fx-course-service:
    - Abrir el proyecto en su IDE
    - Verificar la configuración en application.properties
    - Ejecutar FutureXCourseAppApplication
4. Configurar y ejecutar fx-catalog-service:
    - Abrir el proyecto en su IDE
    - Verificar la configuración en application.properties
    - Ejecutar FutureXCourseCatalogApplication
5. Comprender la interacción entre servicios:
    - Observar cómo fx-catalog-service descubre fx-course-service a través de Eureka:

```
InstanceInfo instanceInfo = client.getNextServerFromEureka("fx-course-service", false);
String courseAppURL = instanceInfo.getHomePageUrl();
```

    - Notar cómo se realiza la llamada al servicio de cursos:

```
RestTemplate restTemplate = new RestTemplate();
courses = restTemplate.getForObject(courseAppURL, String.class);
```

6. Probar los endpoints:
    - Acceder a http://localhost:8002/ para ver el catálogo
    - Acceder a http://localhost:8002/catalog para ver los cursos
    - Acceder a http://localhost:8002/firstcourse para ver el primer curso
7. Observar el panel de Eureka:
    - Acceder a http://localhost:8761/ para ver los servicios registrados

Parte 2: Implementación de Hystrix (Circuit Breaker)

Guía paso a paso:

1. Agregar dependencia de Hystrix en fx-catalog-service:
    - Añadir en pom.xml:

```
<dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-starter-netflix-hystrix</artifactId>
</dependency>
```

2. Habilitar Hystrix en la aplicación:
    - Añadir @EnableCircuitBreaker en FutureXCourseCatalogApplication
3. Implementar métodos de fallback:
    - Crear métodos que se ejecutarán cuando falle la llamada al servicio de cursos
4. Aplicar Hystrix a los métodos del controlador:
    - Usar @HystrixCommand en los métodos de CatalogController
    - Especificar el método de fallback
5. Configurar propiedades de Hystrix:
    - Ajustar timeouts, tamaño de ventana, etc. en application.properties
6. Probar el Circuit Breaker:
    - Detener fx-course-service
    - Acceder a los endpoints de fx-catalog-service y observar el comportamiento

Entregable:

1. Código Fuente de implementación de Hystrix
2. Presentación y demostración

Criterios de Calificación:

1. Comprensión del Service Discovery (40%)
    - Explicación clara de cómo fx-catalog-service descubre y se comunica con fx-course-service
    - Demostración de la funcionalidad a través de los endpoints
2. Implementación de Hystrix (60%)
    - Correcta configuración de Hystrix en el proyecto
    - Implementación adecuada de métodos de fallback
    - Uso apropiado de @HystrixCommand en los métodos del controlador
    - Configuración adecuada de propiedades de Hystrix
    - Demostración exitosa del funcionamiento del Circuit Breaker
    - Explicación clara de cómo Hystrix mejora la resiliencia del sistema